1장
리액트 시작하기

1장 리액트 시작하기

1.1 리액트 개요

1.2 개발 환경 설정하기

1.3 리액트 애플리케이션 만들기

1.1 리액트 개요

- **리액트(React): UI를 구축할 때 가장 널리 사용하는 자바스크립트 기반 프런트엔드 라이브러리**

1.1.1 리액트의 핵심 철학

- 컴포넌트 기반 아키텍처: UI를 작고 재사용 가능한 컴포넌트로 나누어 개발
- 가상 DOM: 변경 사항을 먼저 반영한 뒤 실제 DOM의 변경 부분만 최소한으로 업데이트하는 방식
- 선언적 프로그래밍: 상태만 변경하면 UI가 자동으로 갱신되어 DOM 조작 없이 간결하고 직관적인 코드를 작성

1.1 리액트 개요

1.1.2 리액트의 특징

- 단방향 데이터 흐름: 데이터가 부모에서 자식으로만 흐르고, 자식은 부모의 상태를 직접 변경할 수 없는 구조
- JSX(JavaScript XML) 사용: 리액트에서 사용하는 문법 확장으로, HTML과 유사한 형태로 UI를 정의할 수 있음
- 상태 관리 도구 제공: 상태를 효과적으로 관리할 수 있도록 다양한 기능의 훅(Hooks)을 제공
- 풍부한 생태계: 방대한 생태계를 기반으로 다양한 도구와 라이브러리를 함께 사용

1.1 리액트 개요

※ React Hooks(리액트 훅)

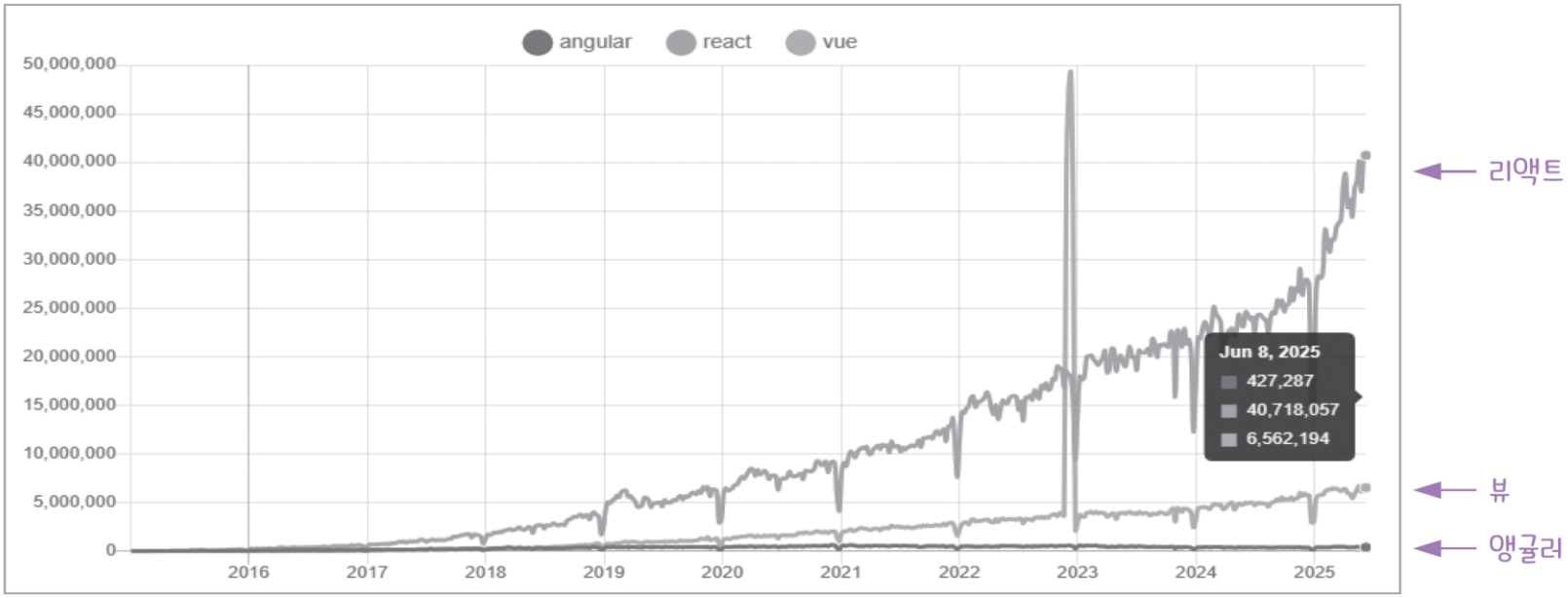
- React 16.8 버전에서 도입된 기능
- 함수형 컴포넌트(Function Component)에서도 상태(State)와 생명주기(Lifecycle) 기능을 사용할 수 있도록 해주는 특별한 함수를 의미
- 함수형 컴포넌트에서 상태 관리와 사이드 이펙트 처리 등의 기능을 사용할 수 있게 해주는 React의 특수 함수
  - 클래스 없이 상태 관리 가능
- Hook이라는 단어는 영어로 “걸다”, “연결하다”라는 의미를 가짐.
- React 내부 기능(함수형 컴포넌트)에 연결(hook) 하여 상태(state), 생명주기(lifecycle), context 등의 기능을 사용할 수 있도록 하는 것.

1.1 리액트 개요

1.1.3 리액트를 배워야 하는 이유

- 여러 해 동안 꾸준히 가장 많은 사용량을 기록
- 사용량도 매년 증가하고 있음

▶ npm trends의 프런트엔드 라이브러리 다운로드 수 비교(출처: https://npmtrends.com)



1.1 리액트 개요

- npm(Node Package Manager)
  - JavaScript 프로그램에서 필요한 라이브러리와 패키지를 설치하고 관리하는 도구
  - 즉, 프로그램을 만들 때 필요한 외부 코드(패키지)를 쉽게 다운로드하고 관리할 수 있도록 해주는 패키지 관리 시스템(package manager)
  - npm이 하는 역할
    - (1) JavaScript 패키지 저장소 제공
    - (2) 패키지 설치 및 관리
    - (3) 프로젝트 의존성 관리

1.1 리액트 개요

▶ **리액트를 사용하는 기업(출처: https://www.codenary.co.kr/techstack/detail/reactjs)**

CHAI	차이코퍼레이션	42dot	42dot	kakao ENTERTAINMENT	카카오엔터테인먼트	의식주컴퍼니	의식주컴퍼니
kakao enterprise	카카오엔터프라이즈	요기요	위대한상상	두나무	두나무		원티드랩
MUSINSA	무신사	coupang	쿠팡	toss	비바리퍼블리카	yanolja	야놀자
HYPERCONNECT	하이퍼커넥트	kakaomobility	카카오모빌리티		뱅크샐러드	kakao	카카오
우아한형제들	우아한형제들		스켈터랩스	LINE	라인	NAVER	네이버
SNOW	스노우	WEBTOON	네이버웹툰	spoqa	스포카	vcnc	브이씨앤씨

1.1 리액트 개요

1.1.4 리액트의 발전(1)

1. 리액트의 등장(2013): 컴포넌트와 가상 DOM의 혁신
2. 리액트 16(2017): 혁신의 전환점, 안정성과 유연성 확보
3. 리액트 18(2022): 동시성과 성능의 진화
   - 동시성 렌더링(concurrent rendering): 여러 작업을 동시에 처리해 UI 반응성 개선
   - 자동 배치(auto batching): 여러 상태 업데이트를 자동으로 하나의 렌더링으로 묶어 성능 최적화 – 한번에 묶어서 렌더링 횟수를 줄이는 것.
   - 서버 컴포넌트(server components): 서버에서 일부 UI를 렌더링해 초기 로딩 속도 개선

1.1 리액트 개요

1.1.4 리액트의 발전(2)

4. 리액트 19(2024): 새로운 가능성의 시작

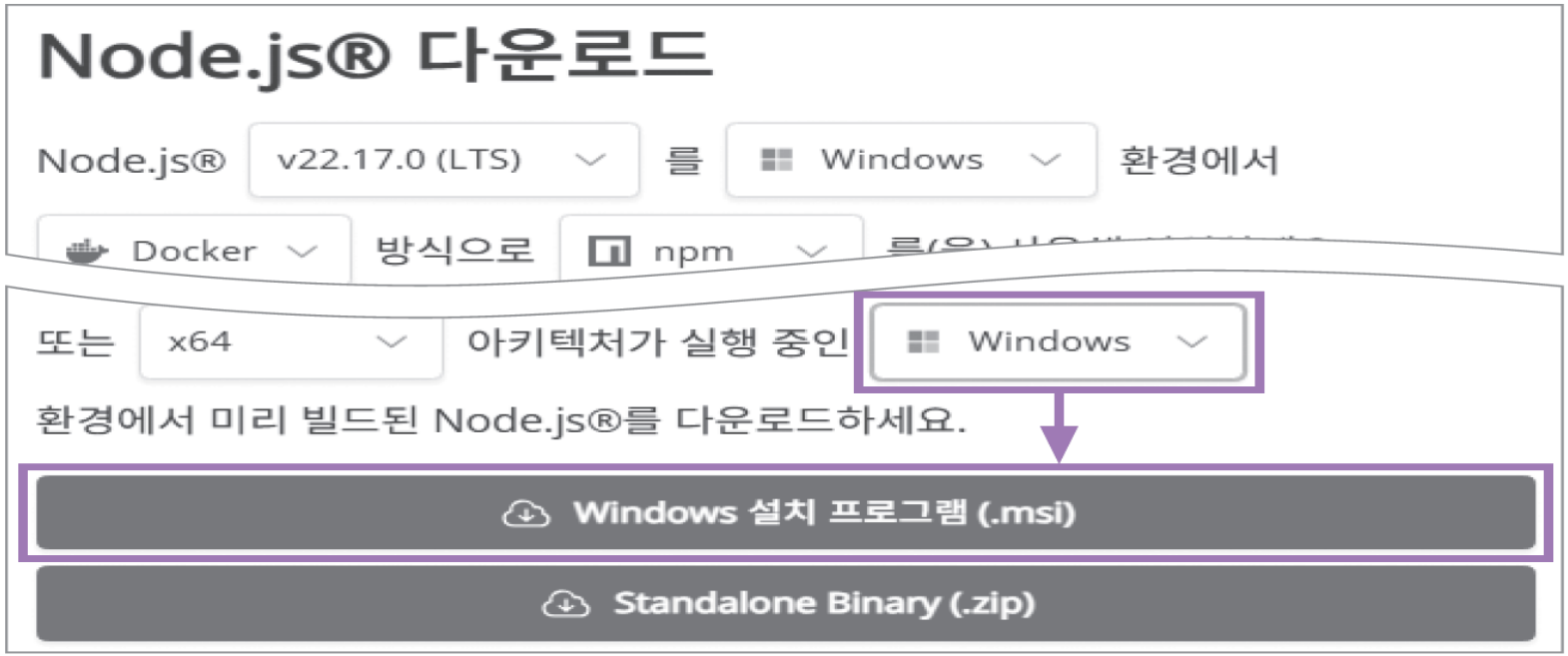
- React 19가 단순히 렌더링 최적화만 한 것이 아니라, **개발자가 자주 겪는 불편한 작업들을 React 안으로 더 자연스럽게 끌어들였기 때문.**
- React 19는 다음 질문에 대한 답을 더 잘 제공하기 시작
  - **폼 제출할 때 로딩 상태를 어떻게 관리하지?**
  - **서버에 요청 보내는 동안 사용자에게 빠른 반응을 어떻게 보여주지?**
  - **<title>이나 <meta>를 더 자연스럽게 다루려면?**
  - **서버와 클라이언트가 섞인 최신 React 구조를 더 쉽게 쓰려면?**
- **즉, React 19는 "React가 UI 라이브러리에서 더 넓은 애플리케이션 실행 환경으로 확장되는 시작점" 처럼 볼 수 있음.**

1.2 개발 환경 설정하기

1.2.1 Node.js 설치

- **Node.js 다운로드 페이지**
  https://nodejs.org/ko/download

▶ **운영체제에 맞는 설치 파일 다운로드**



1.2 개발 환경 설정하기

- **Node.js 설치 확인**

```
C:₩WINDOWS₩system32₩cmd.exe
Microsoft Windows [Version 10.0.19045.5965]
(c) Microsoft Corporation. All rights reserved.

C:\Users\gilbut>node -v
v22.17.0

C:\Users\gilbut>_
```

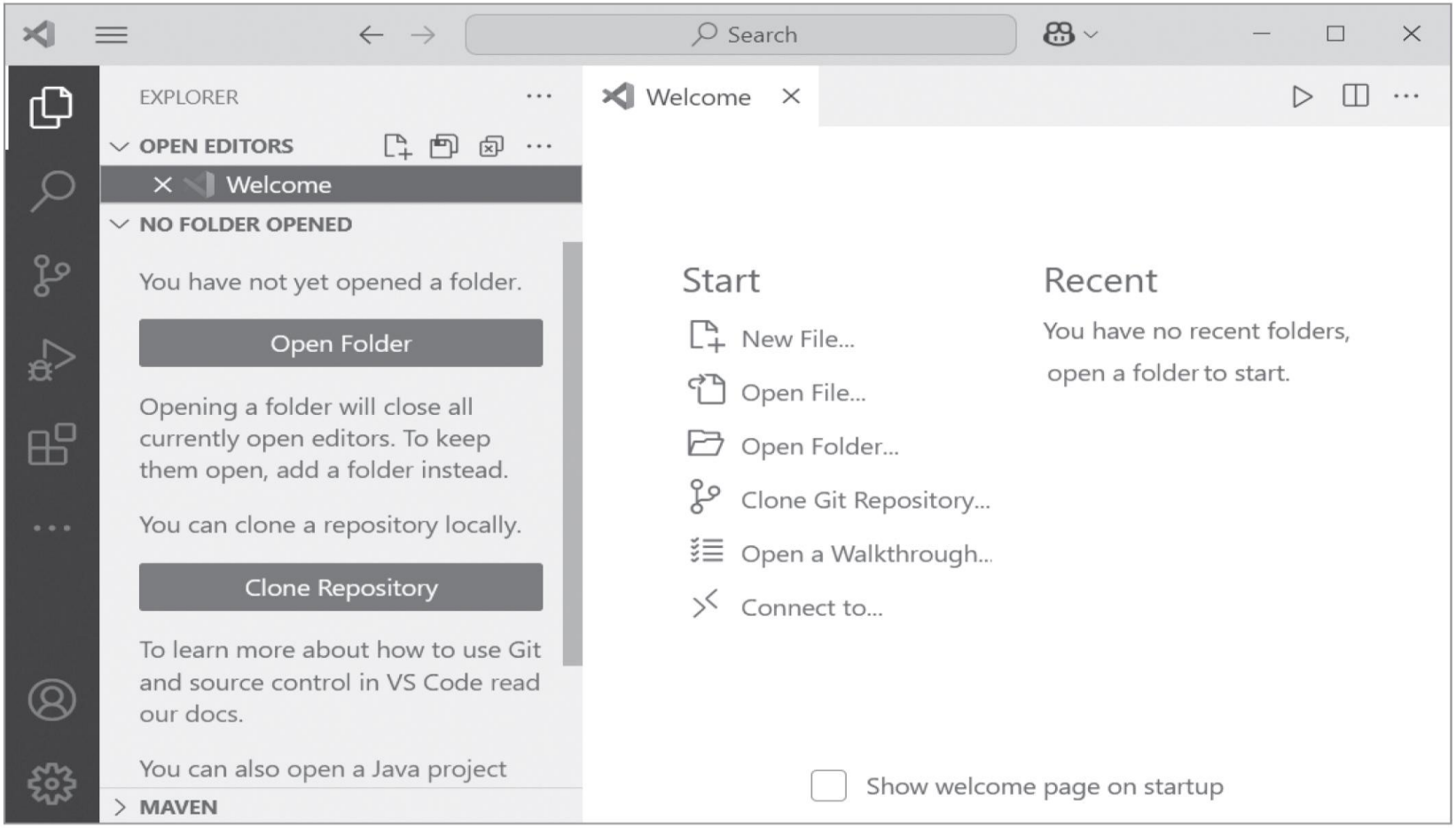
1.2 개발 환경 설정하기

1.2.2 VSCode 설치

▶ **VSCode 설치 프로그램 다운로드**

1.2 개발 환경 설정하기

▶ VSCode 초기 화면



1.2 개발 환경 설정하기

1.2.3 VSCode 익스텐션 설치

- **익스텐션(extension): 개발에 필요한 기능을 추가해주는 작은 프로그램**

● ES7+ React/Redux/React-Native snippets

- **리액트에서 자주 사용하는 코드 조각(스니핏)을 저장해 두고 , 단축어처럼 빠르게 입력할 수 있도록 도와주는 도구**

● ESLint

- **리액트 문법을 작성할 때 발생할 수 있는 오류를 실시간으로 검사해주는 도구로써 이를 사용하면 실수하기 쉬운 문법 오류를 빠르게 확인하고 수정 가능함. ESLint는 개인 프로젝트 뿐만 아니라 팀 단위 개발에서도 필요한 익스텐션.**

1.2 개발 환경 설정하기

1.2.3 VSCode 익스텐션 설치

●Tailwind CSS IntelliSense

- Tailwind CSS를 학습하는데 필요한 익스텐션
- Tailwind CSS에서 제공하는 유틸리티 클래스를 보다 편리하게 사용할 수 있도록 도와주는 다양한 기능을 활용 가능함
- 자동 완성 : 유틸리티 클래스 이름을 입력할 때 Tailwind CSS에서 제공하는 모든 클래스를 실시간으로 추천함
- 클래스 유효성 검사 : 오타가 있거나 더 이상 사용하지 않는 (deprecated) 클래스를 입력하면 에디터에서 즉시 경고 메시지를 표시함
- 문서 링크 제공 : 클래스 이름 위에 마우스 커서를 올리면 해당 클래스가 어떤 스타일을 적용하는지 요약된 설명과 함께 공식 문서로 이동할 수 있는 링크가 포함된 툴팁을 표시함

1.2 개발 환경 설정하기

●그 외 익스텐션

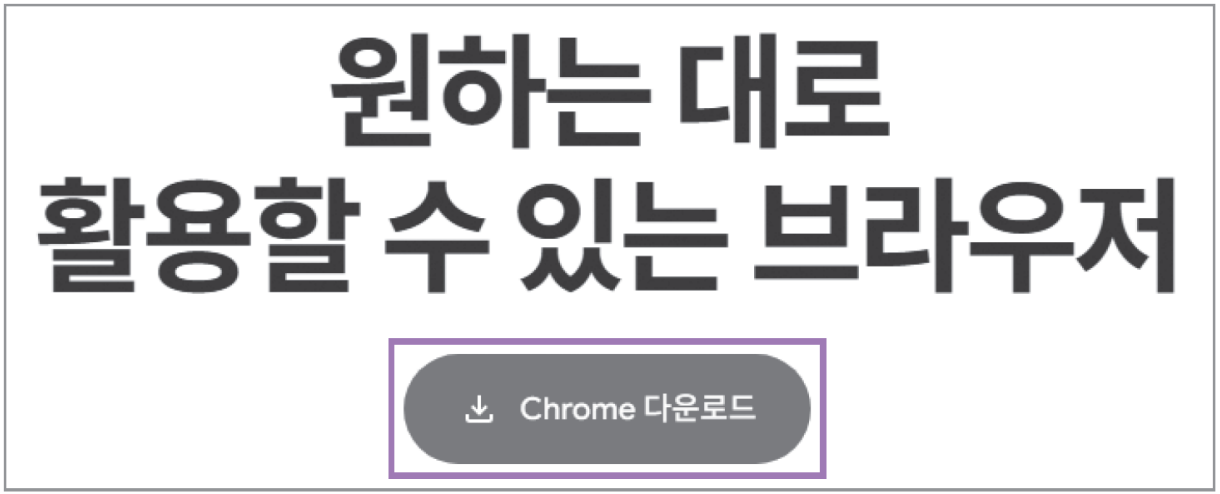
1. Live Server(제작자: Ritwick Dey)
2. Prettier - Code formatter(제작자: Prettier)
3. Auto Rename Tag(제작자: Jun Han)
4. HTML to CSS autocompletion(제작자: solnurkarim)
5. HTML CSS Support(제작자: ecmel)
6. Code Runner(제작자: Jun Han)
7. Material Icon Theme(제작자: Philipp Kief)
8. Korean Language Pack for Visual Studio Code(제작자: Microsoft)

1.2 개발 환경 설정하기

1.2.4 웹 브라우저 설치

- 크롬 공식 사이트
  https://www.google.co.kr/chrome

▶ 크롬

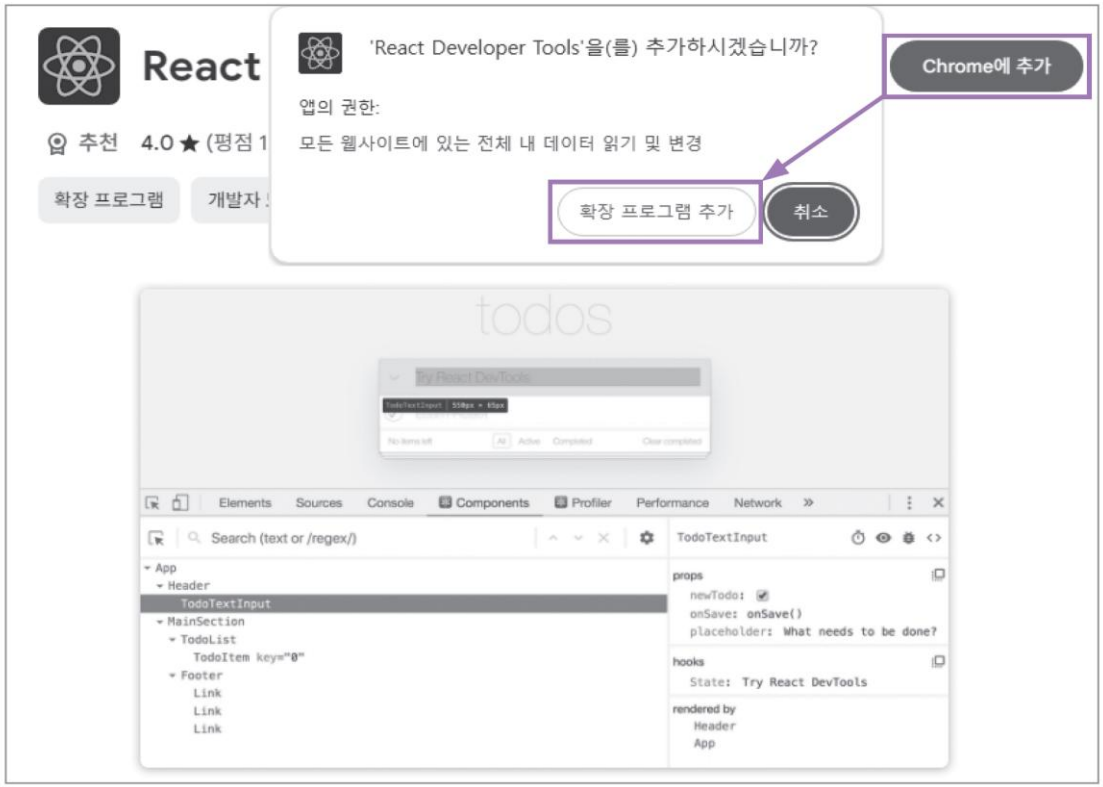


1.2 개발 환경 설정하기

●확장 프로그램 설치

- **React Developer Tools: 컴포넌트 구조, 상태, props 등 리액트 내부 구조를 시각적으로 확인**

▶ **React Developer Tools 을 크롬에 추가**



1.3 리액트 애플리케이션 만들기

- React 프로젝트는 처음부터 모든 파일과 설정을 직접 만드는 것이 아니라 기본 구조가 미리 설정된 템플릿(보일러플레이트)을 사용해 빠르게 시작하는 것이 일반적임.
- CRA(Create React App): 페이스북에서 만든 공식 프로젝트 생성 도구, 안정성과 방대한 생태계가 강점. 현재는 공식적으로 deprecated(표준적인 사용 자제 권고) 상태
- Vite: 빠르고 가벼운 개발 도구, 원래 Vue.js용으로 개발. 기존 도구와 비교해 더 빠른 개발 서버와 빌드 시간을 제공. 설정도 간단하며 다양한 프레임워크를 지원. 최근 선호하는 추세임.

1.3 리액트 애플리케이션 만들기

1.3.1 Vite로 프로젝트 생성하기

▶ 실습 폴더 선택

File Edit Selection ··· vite-react

EXPLORER

OPEN EDITORS

VITE-REACT

Do you trust the authors of the files in this folder?

Code provides features that may automatically execute files in this folder.

If you don't trust the authors of these files, we recommend to continue in restricted mode as the files may be malicious. See our docs to learn more.

D:\react\vite-react

Trust the authors of all files in the parent folder 'react'

Yes, I trust the authors — *Trust folder and enable all features*

No, I don't trust the authors — *Browse folder in restricted mode*

Start Debugging F5

Toggle Terminal Ctrl + `

1.3 리액트 애플리케이션 만들기

- **npm create: npm을 사용해 프로젝트 생성 도구를 실행하는 명령어**
- **vite@latest: Vite의 최신 버전을 사용하겠다는 의미**
- **.(마침표): 현재 폴더에 프로젝트를 생성하겠다는 뜻**

```
TERMINAL
npm create vite@latest .
```

▶ **터미널에 명령어 입력**

```
TERMINAL
Microsoft Windows [Version 10.0.19045.5965]
(c) Microsoft Corporation. All rights reserved.

D:\react\vite-react>npm create vite@latest .
```

1.3 리액트 애플리케이션 만들기

- **npm install: 프로젝트에 필요한 의존성 패키지 설치 명령어**
- **npm run dev: 개발 서버를 실행해 리액트 애플리케이션을 웹 브라우저에서 확인할 수 있게 함**

TERMINAL

```
npm install
npm run dev
```

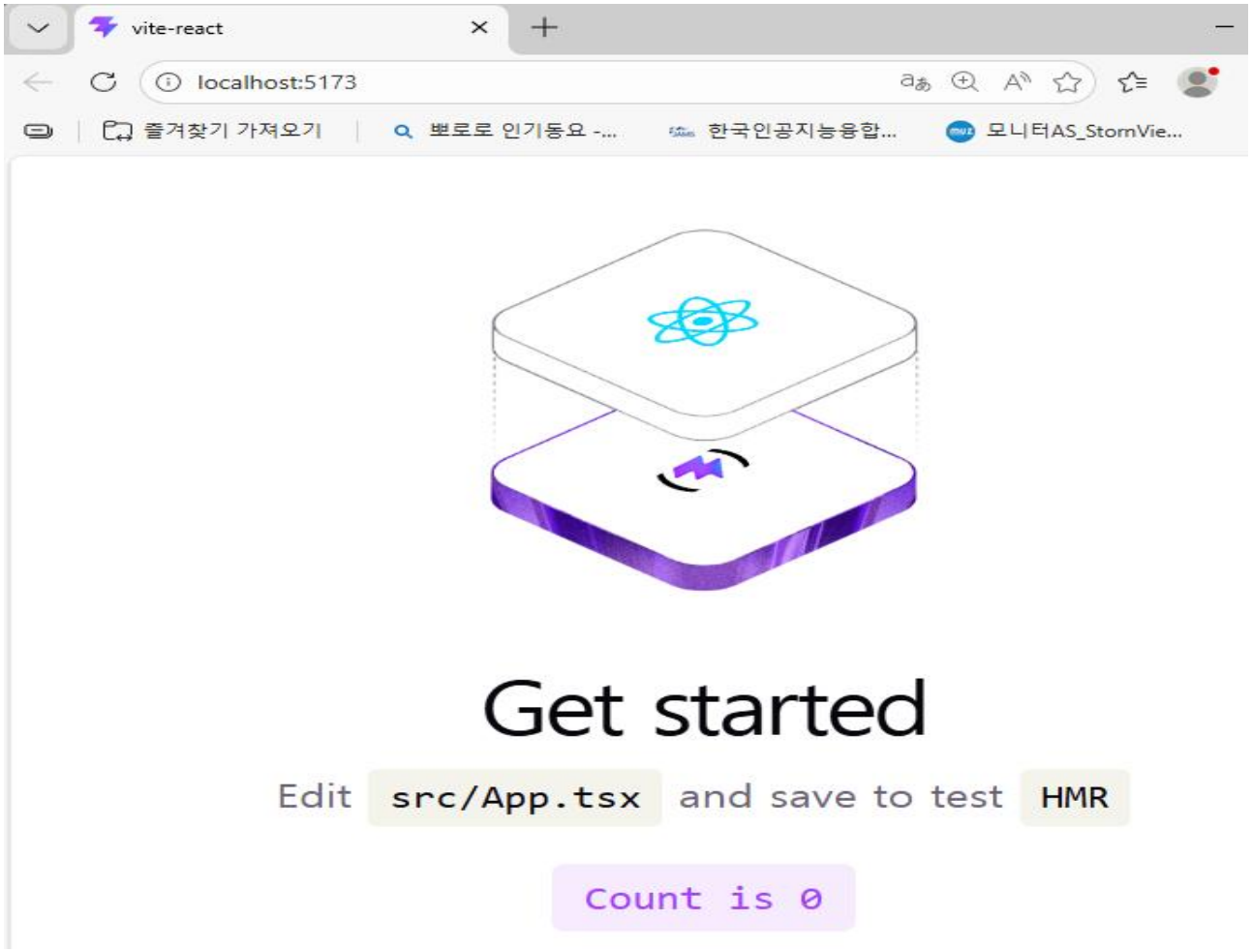
TERMINAL

```
VITE v7.0.4  ready in 404 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

1.3 리액트 애플리케이션 만들기

▶ **생성한 리액트 애플리케이션 화면**

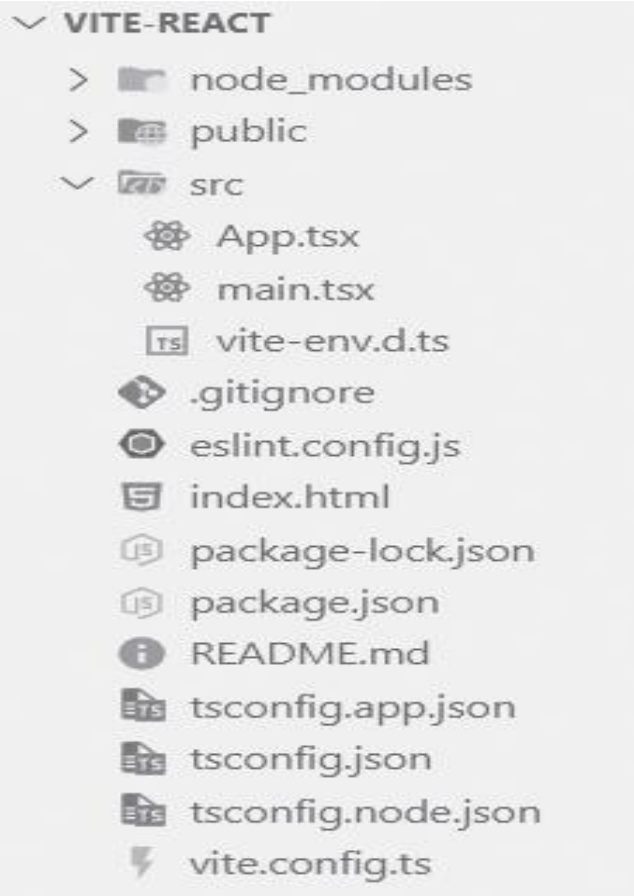


1.3 리액트 애플리케이션 만들기

1.3.2 불필요한 폴더와 파일 삭제 및 수정하기

▶ **생성한 프로젝트의 구조**

▶ **프로젝트의 최종 구조**



1.3 리액트 애플리케이션 만들기

1.3.3 기본 구조 살펴보기

- 프로젝트에 있는 여러 폴더와 파일중에서 애플리케이션을 실행하는데 특히 중요한 파일은 package.json , index.html, main.tsx, App.tsx 임

- package.json 파일: 프로젝트의 핵심 설정 파일. 이 파일에는 프로젝트 이름과 버전, 사용하는 라이브러리, 실행할 수 있는 명령어(스크립트) 등의 정보가 담겨 있음.
  - name : React 애플리케이션의 이름. 보통 폴더 이름과 동일하게 생성됨

1.3 리액트 애플리케이션 만들기

1.3.3 기본 구조 살펴보기

- **package.json 파일: 프로젝트의 핵심 설정 파일.**
  - private : 패키지를 외부(npm 레지스트리)에 공개(배포)할지 여부를 설정. 대부분의 프로젝트에서는 true로 설정해 비공개로 유지함. 공개하려면 private 값을 false로 설정하거나 아예 제거하면 됨. 다만 실제로 npm 레지스트리에 공개되는 것은 npm publish 명령을 실행할 때이며, private 값은 실수로 배포되는 것을 막는 안전장치 역할을 함
  - version : 프로젝트 버전을 의미. 형식은 major.minor.patch(주버전, 부버전, 수정버전) 임

1.3 리액트 애플리케이션 만들기

1.3.3 기본 구조 살펴보기

- **package.json 파일: 프로젝트의 핵심 설정 파일.**
  - scripts : 터미널에서 실행할 수 있는 명령어를 정의하는 부분. 여기에 등록한 명령어는 npm run 명령어로 실행 가능함. (예) npm run dev
  - dependencies : 리액트 애플리케이션을 실행하는 데 필요한 의존성 라이브러리 목록. 여기에 정의한 라이브러리는 프로덕션(production, 애플리케이션을 배포하고 서비스하는 단계) 환경에서도 사용함

1.3 리액트 애플리케이션 만들기

1.3.3 기본 구조 살펴보기

- **package.json 파일: 프로젝트의 핵심 설정 파일.**
  - **devDependencies**
    - 리액트 애플리케이션을 개발할 때 필요한 도구 목록.
    - 개발 과정에서만 필요한 패키지들을 정의하는 항목. 즉, 애플리케이션 실행(runtime)에는 필요 없고 개발·빌드·테스트 단계에서만 사용하는 라이브러리를 관리함.
    - (예) 타입스크립트, ESLint 등

1.3 리액트 애플리케이션 만들기

1.3.3 기본 구조 살펴보기

- **package.json 파일: 프로젝트의 핵심 설정 파일.**
  - 이 항목들 중에서 특히 scripts, dependencies , devDependencies 항목이 중요함.
    - (예) 이전에 개발서버를 실행할 때 다음 명령어를 입력하였음

TERMINAL

```
npm run dev
```

    - 이 명령어는 package.json 파일의 scripts 항목 안에 다음 같은 설정이 있기 때문에 실행됨. 즉, npm run dev 명령 [illegible] e 명령을 실행해 개발 서버를 구

```
"scripts" : {
        "dev" : "vite"
}
```

1.3 리액트 애플리케이션 만들기

1.3.3 기본 구조 살펴보기

- **외부 라이브러리(패키지) 설치 명령어**

형식

```
npm install [패키지명] [--save¦--save-dev]
```

- **--save 옵션으로 설치하면 dependencies 에 등록되고 –save-dev 옵션으로 설치하면 devDependencies 에 등록됨**
- **아무 옵션도 넣지 않으면 자동으로 --save 옵션이 적용되어 dependencies 에 등록됨**
- **개발에 필요한 패키지와 배포에 필요한 패키지를 구분해서 설치해야 함**

1.3 리액트 애플리케이션 만들기

●index.html 파일(1)

- 터미널에서 “npm run dev” 명령어를 입력해 개발 서버를 실행하면 Vite 개발 서버는 내부적으로 “index.html” 파일을 자동으로 로드함
- 리액트 애플리케이션을 실행하는 데 필요한 최소한의 HTML 구조가 들어 있음
- 브라우저가 처음 로딩하는 HTML 파일이며, React 애플리케이션이 화면에 렌더링 될 시작 지점(entry point) 역할을 수행
- 는 React 애플리케이션이 화면에 렌더링되는 컨테이너(container)

1.3 리액트 애플리케이션 만들기

●index.html 파일(2)

- React는 HTML 전체를 직접 작성하지 않고 JavaScript로 UI를 생성함. 이때 React가 생성한 UI가 삽입되는 위치가 바로 root div 임
- main.tsx에서 동일한 ID(“root”)로 연결해야 함.
- 맨 아랫 부분의 다음 코드는
  <script type=“module” src=“/src/main.tsx”><script>
  리액트 애플리케이션의 시작점인 main.tsx를 불러오는 코드. type=“module” 속성은 해당 파일 내부에서 모듈 기능(import, export)을 사용할 수 있게 웹브라우저에 알려주는 역할 수행. 이 스크립트를 통해 리액트 컴포넌트, 상태, 라우터 등 애플리케이션 전체가 실행됨

1.3 리액트 애플리케이션 만들기

●main.tsx 파일

- index.html 파일에서 <script type="module" src="/src/main.tsx"> 로 main.tsx 파일을 불러오면 그때부터 리액트 코드가 실행됨
- React 애플리케이션의 시작 파일(Entry Point)
- 리액트 애플리케이션을 초기화하고 구성하는 역할을 수행
- React 앱을 브라우저의 HTML DOM에 연결하고 화면에 렌더링하는 역할 수행
- main.tsx는 React 앱을 시작하고<App /> 컴포넌트를 HTML(index.html) 의 root div에 렌더링하는 파일

1.3 리액트 애플리케이션 만들기

●main.tsx 파일

```
import { StrictMode } from 'react'
```

- React 에서 StrictMode 컴포넌트를 가져오는 코드
- StrictMode의 역할
  - StrictMode는 개발 환경에서 코드의 문제를 검사하는 도구
  - 리액트에서 코드 오류나 비효율적 사용을 알려주는 도구
- 주요 기능
  - 잠재적인 버그 탐지
  - 오래된 React API 사용 경고
  - 안전하지 않은 코드 패턴 확인
  - 개발 단계에서 추가 검사 수행

1.3 리액트 애플리케이션 만들기

- main.tsx 파일

```
<StrictMode>
  <App />
</StrictMode>
```

- React 에서 App 전체 컴포넌트를 검사
- 중요한 특징
  - 개발 모드에서만 동작
  - 실제 배포(build)에서는 영향 없음

1.3 리액트 애플리케이션 만들기

- main.tsx 파일

```
import { createRoot } from 'react-dom/client'
```

- 이 코드는 ReactDOM 에서 createRoot 함수를 가져오는 코드
- ReactDOM의 역할
  - React에서 만든 UI를 브라우저 DOM에 렌더링하는 라이브러리
- createRoot 의미
  - createRoot()는 React 18에서 도입된 새로운 렌더링 방식

1.3 리액트 애플리케이션 만들기

- main.tsx 파일

```
import { createRoot } from 'react-dom/client'
```

- createRoot 의미
  - 예전 방식 (React 17)

```
ReactDOM.render(<App />, document.getElementById('root'))
```

  - React 18 방식

```
createRoot(container).render(<App />)
```

    - 이 방식은 Concurrent Rendering 지원
    - 성능 개선 / 비동기 렌더링 최적화를 위해 도입됨

1.3 리액트 애플리케이션 만들기

- **main.tsx 파일**

```
import App from './App.tsx'
```

  - **App 컴포넌트를 가져오는 코드**
  - **App 컴포넌트는 React 애플리케이션의 최상위 컴포넌트로써 전체 UI의 시작 컴포넌트임**

- **React 애플리케이션 구조**

```
App
  ├─ Header
  ├─ Main
  └─ Footer
```

1.3 리액트 애플리케이션 만들기

- main.tsx 파일

```
createRoot(document.getElementById('root')!)
```

  - HTML에서 root 요소를 찾아 React 앱을 연결

- HTML 파일 (index.html)

```
<div id="root"></div>
```

- 이 코드들은 다음과 같이 연결됨

```
HTML DOM
    |
    └── root div
            |
            └── React 앱 연결
```

1.3 리액트 애플리케이션 만들기

main.tsx 파일

- 브라우저 DOM에서 root 요소 찾기

```
document.getElementById('root')
```

- 결과로써 index.html 에 있는 아래 태그를 찾아서 연결함

```
<div id="root"></div>
```

- 아래 코드에서 "!" 의 의미 (TypeScript)

```
document.getElementById('root')!
```

- ! 는 Non-null Assertion Operator : "이 값은 null이 아니다"
- TypeScript에게 "root 요소는 반드시 존재한다"라는 것을 보장함

1.3 리액트 애플리케이션 만들기

●main.tsx 파일

- **render() 함수**

```
.render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

✓ **React 컴포넌트를 실제 화면에 렌더링 함**

- **렌더링되는 구조**

```
root
 |
 └── StrictMode
        |
        └── App
               |
               └── React UI
```

1.3 리액트 애플리케이션 만들기

main.tsx 파일

- render() 함수

```
.render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

- <App />
  - React의 JSX 컴포넌트 문법
  - App 컴포넌트를 실행하여 UI 생성하라는 의미
  - (예)

```
function App() {
  return <h1>Hello React</h1>
}
```

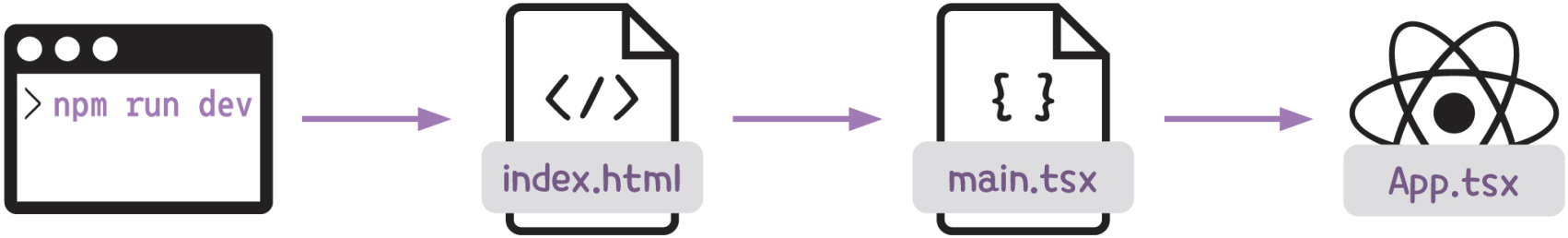
1.3 리액트 애플리케이션 만들기

- React 앱이 실행되는 전체 과정의 흐름

```
1 브라우저가 index.html 로드
        |
2 main.tsx 실행
        |
3 root div 찾기
        |
4 React Root 생성
        |
5 App 컴포넌트 렌더링
        |
6 화면에 UI 출력
```

1.3 리액트 애플리케이션 만들기

- **리액트 애플리케이션의 전체 실행 과정**
  1. **npm run dev로 개발 서버를 실행합니다.**
  2. **개발 서버는 index.html 파일을 엽니다.**
  3. **index.html 파일은 main.tsx 파일을 불러옵니다.**
  4. **main.tsx 파일에서 App 컴포넌트를 읽어 실행합니다.**
  5. **App 컴포넌트가 웹 브라우저 화면에 표시됩니다.**



1.3 리액트 애플리케이션 만들기

●React 앱의 구조

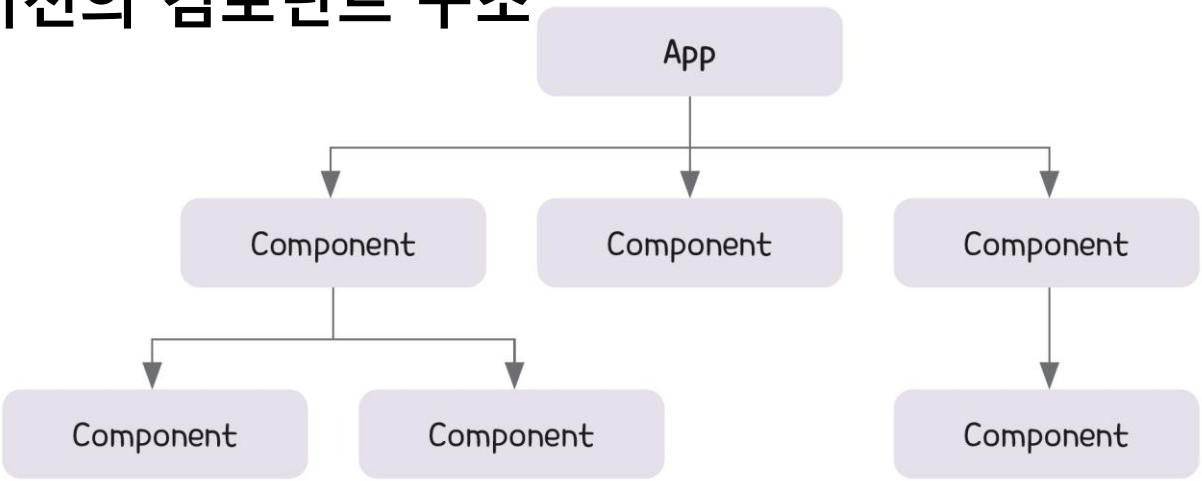
```
index.html
    |
    └── <div id="root"></div>
                |
                ▼
main.tsx
    |
    └── createRoot(...)
            |
            ▼
<App />
            |
            ▼
React UI
```

1.3 리액트 애플리케이션 만들기

●App.tsx

- 루트 컴포넌트(root component): 애플리케이션의 시작점이 되는 가장 중요한 컴포넌트
- 트리(tree) 구조: 루트 컴포넌트를 중심으로 하위 컴포넌트들이 가지처럼 뻗어 나가는 구조

▶ 리액트 애플리케이션의 컴포넌트 구조



2장
JSX 개요

2장 JSX 개요

2.1 JSX란
2.2 JSX의 문법적 특징

2.1 JSX란

- JSX = JavaScript + XML
- JSX는 ‘JavaScript XML’의 줄임말
- XML 구문에 자바스크립트 코드를 결합하는 용도로 만들어진 구문
- '자바스크립트 언어를 확장한다’
  => 표준 ESNext 자바스크립트나 타입스크립트 문법 자체에는 JSX 구문이 없다는 의미
- 리액트 개발 팀은 마치 JSX 구문이 표준 자바스크립트 문법에 포함되어 있는 것처럼 동작하도록 설계

2.1 JSX란

- **JSX(JavaScript XML): 자바스크립트 코드 안에서 HTML과 비슷한 문법을 사용해 UI를 정의할 수 있도록 도와주는 문법 확장(syntactic extension)**

- **코드 비교**

HTML

```
<div>
  <h1>Hello, React!</h1>
</div>
```

JSX

```
<div>
  <h1>Hello, React!</h1>
</div>
```

리액트

```
React.createElement(
  'div',
  null,
  React.createElement('h1', null, 'Hello, React!')
);
```

- **JSX 코드가 HTML 처럼 보이지만 JSX 문법**
- **React는 JSX를 내부적으로 createElement()함수 호출 코드로 바꾸어서 실행함**

2.1 JSX의 예

```
03 const rootVirtualDOM = (
04   <ul>
05     <li>
06       <a href="http://www.google.com" target="_blank">
07         <p>go to Google</p>
08       </a>
09     </li>
10   </ul>
11 )
12
13 const root = ReactDOM.createRoot(document.getElementById('root') as HTMLElement)
14 root.render(rootVirtualDOM)
```

자바스크립트가 아님!!!

2.1 JSX란

JSX

```
<div>
  <h1>Hello, React!</h1>
</div>
```

```
export default function App() {
  return (
    <div>
      <h1>Hello, React!</h1>
    </div>
  );
}
```

JSX

- ** 와 <h1>태그는 JSX 코드이고 함수 정의나 return 구문은 JavaScript 임**

2.2 JSX의 문법적 특징

2.2.1 하나의 루트 요소로 반환하기

잘못된 코드: 두 개 이상의 루트 요소 반환

```
<div>첫 번째 영역</div>
<div>두 번째 영역</div>
```

올바른 코드: 하나의 루트 요소 반환

```
<div>
  <div>첫 번째 영역</div>
  <div>두 번째 영역</div>
</div>
```

2.2 JSX의 문법적 특징

- **프래그먼트(Fragment): 여러 요소를 하나로 묶어주는 가상의 태그**

기본 문법

```
<React.Fragment>
  <div>첫 번째 영역</div>
  <div>두 번째 영역</div>
</React.Fragment>
```

단축 문법

```
<>
  <div>첫 번째 영역</div>
  <div>두 번째 영역</div>
</>
```

2.2 JSX의 문법적 특징

2.2.2 모든 태그 닫기

- 빈 태그(self-closing tag): <br>, <img>, <input>처럼 내용 없이, 시작 태그만 존재하는 경우
- 자체 종료 태그(self-closing tag): 모든 태그는 반드시 열고 닫는 구조를 가져야 하며, 빈 태그도 예외가 아님

JSX

```
<br />
<img />
```

✓ JSX 에서는 빈태그라도
처럼 반드시 자체 종료 태그를 사용해야 함

✓ 즉, 모든 태그는 반드시 열고 닫는 구조를 가져야 하며, 빈 태그도 예외는 아님

2.2 JSX의 문법적 특징

2.2.3 태그 속성은 카멜 케이스로 작성하기

- **카멜 케이스(camelCase): 여러 단어로 이루어진 이름을 붙일 때, 첫 번째 단어는 첫 글자를 소문자, 두 번째 단어부터는 첫 글자를 대문자로 표기하는 방식**

▶ **HTML과 JSX에서 서로 이름이 다른 속성(1)**

HTML 속성명	JSX 속성명
class	className
for	htmlFor
contenteditable	contentEditable
tabindex	tabIndex
readonly	readOnly
maxlength	maxLength
colspan	colSpan
rowspan	rowSpan

2.2 JSX의 문법적 특징

2.2.3 태그 속성은 카멜 케이스로 작성하기

▶ **HTML과 JSX에서 서로 이름이 다른 속성**

HTML 속성명	JSX 속성명
frameborder	frameBorder
usemap	useMap
accesskey	accessKey
enctype	encType
accept-charset	acceptCharset
inputmode	inputMode
autoplay	autoPlay
formnovalidate	formNoValidate

2.2 표현식과 실행문, 그리고 JSX

- **표현식(expression)**
  - 타입스크립트(혹은 ESNext 자바스크립트)에서 return **키워드 없이 값만을 반환**하는 구문
- 실행문(execution statement)
  - switch/case 문이나 for 문처럼 값을 반환하지 않는 구문 (대부분의 자바스크립트 구문)
- JSX 문에 실행문을 사용하여 발생한 오류 예:

```
3   const div = <div>{console.log('hello')}</div>
4
```

문제 1 디버그 콘솔 터미널

TS index.tsx src 1

'void' 형식은 'ReactNode' 형식에 할당할 수 없습니다. ts(2322) [3, 18]

그림 2-16 JSX 문에 console.log 호출이 있어서 발생한 오류

2.2 JSX의 문법적 특징

2.2.4 표현식은 중괄호 안에서 사용하기

- **중괄호({}) 안에 자바스크립트 표현식을 넣어 사용할 수 있음**
- **JSX 내에서 자바스크립트 표현식을 사용하면 동적 데이터**

다음 코드는 {10 + 20}이라는 자바스크립트 표현식을 계산한 뒤 결과인 30을 화면에 표시합니다.

```
<h1>result: {10 + 20} </h1>
```

그림 2-2 자바스크립트 표현식 사용 결과

result: 30

2.2 JSX의 문법적 특징

2.2.4 표현식은 중괄호 안에서 사용하기

JSX 안에서 삼항 연산자(조건 ? 참일 때 : 거짓일 때)도 사용할 수 있습니다. 다음 코드는 조건 10 < 20이 참(true)이므로 '10이 더 작다'가 화면에 표시됩니다.

```
<h1>{10 < 20 ? "10이 더 작다" : "20이 더 크다"}</h1>
```

그림 2-3 삼항 연산자 사용 결과

10이 더 작다

2.2 JSX의 문법적 특징

2.2.4 표현식은 중괄호 안에서 사용하기

논리 연산자 &&를 사용하면 조건이 참일 때만 JSX 요소를 출력할 수 있습니다. 다음 코드에서 첫 번째 조건인 10 > 20은 거짓이므로 <h1>10이 20보다 크다</h1>는 렌더링되지 않습니다. 두 번째 조건인 10 < 20은 참이므로 해당 태그만 화면에 나타납니다.

```
<>
  {10 > 20 && <h1>10이 20보다 크다</h1>}
  {10 < 20 && <h1>10이 20보다 작다</h1>}
</>
```

그림 2-4 논리 연산자 && 사용 결과

10이 20보다 작다

2.2 JSX의 문법적 특징

2.2.5 인라인 스타일은 객체로 지정하기

- 인라인 스타일 : 태그 안에 직접 스타일을 작성하는 방식
- (예) style 속성에 문자열을 할당해 <h1>태그의 텍스트 색상을 red, 배경 색상을 lightgray 로 설정함

```
<h1 style="color: red; background-color: lightgray">Styled Text</h1>
```

2.2 JSX의 문법적 특징

2.2.5 인라인 스타일은 객체로 지정하기

- style 속성에 문자열이 아닌 자바스크립트 객체를 할당
- CSS 속성명은 반드시 카멜 케이스로 작성
  - 속성 이름에 하이픈(-)이 있다면 모두 카멜 케이스로 바꾸어 야 함
    - (ex) background-color은 backgroundColor로 변경

2.2 JSX의 문법적 특징

2.2.5 인라인 스타일은 객체로 지정하기

➢ 앞의 예제를 JSX 로 변경한 코드
(예) style 속성에 문자열을 할당해 <h1>태그의 텍스트 색상을 red, 배경 색상을 lightgray 로 설정함

```
<h1
  style={{
    color: "red",
    backgroundColor: "lightgray",
  }}
>Styled Text
</h1>
```

<실행결과>

Styled Text

2.2 JSX의 문법적 특징

2.2.6 중괄호 안에 주석 작성하기

- 주석(comment): 코드에 설명을 추가하거나 특정 부분을 임시로 비활성화할 때 사용
- JSX에서는 반드시 중괄호 { }안에 /* 주석 내용 */ 형태로 작성해야 함

잘못된 사용 방법

```
<>
  {/* 올바른 주석 */}
  // 잘못된 주석
  /* 잘못된 주석 */
  <div>잘못된 주석 사용 예</div>
</>
```

<실행결과>

```
// 잘못된 주석 /* 잘못된 주석 */
잘못된 주석 사용 예
```

3장
컴포넌트

3장 컴포넌트

3.1 컴포넌트란
3.2 컴포넌트의 종류
3.3 컴포넌트 기초
3.4 컴포넌트와 props 객체
3.5 컴포넌트와 이벤트

3.1 컴포넌트란

- 컴포넌트 - 객체지향 언어 용어로 화면 UI를 담당하는 클래스
- 클래스 컴포넌트와 함수 컴포넌트 두 가지
- 컴포넌트는 리액트가 제공하는 기본 컴포넌트와 리액트 개발자가 필요에 의해 만드는 사용자 컴포넌트 두 가지가 존재
- 리액트 제공 기본 컴포넌트는 소문자로 시작(예:)
- 사용자 컴포넌트는 대문자로 시작하는 낙타등 표기법 사용(예: <HelloWorld/>)

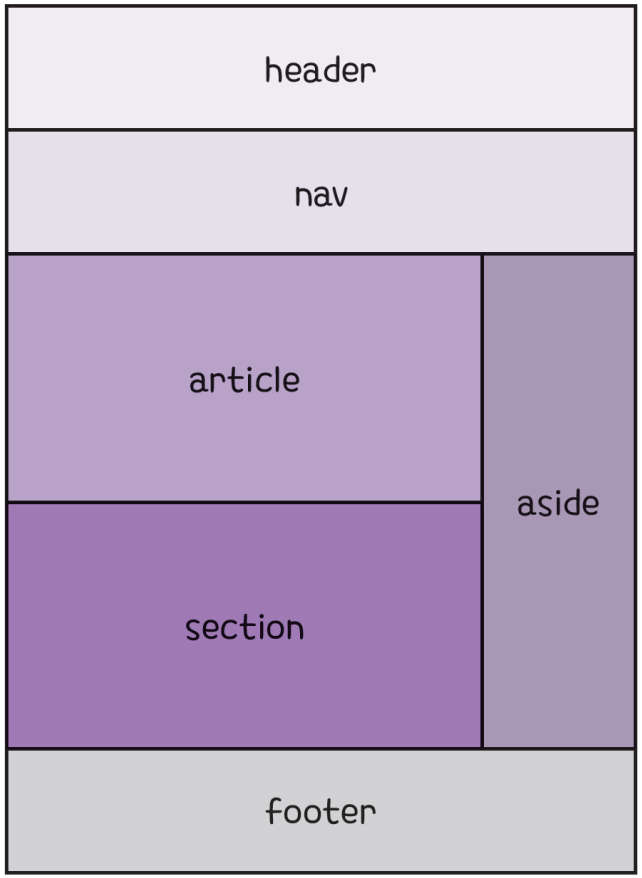
3.1 컴포넌트란

- 컴포넌트(component): UI를 구성하는 독립적이며 재사용 가능한 작은 단위
- React 는 이러한 컴포넌트를 조합해 하나의 애플리케이션을 완성함.

3.1 컴포넌트란

- 추상화(abstraction): 웹 페이지를 기능 단위로 나누는 과정
- 추상화는 복잡한 구조를 단순하게 표현함으로써 전체 시스템을 더 명확하게 이해하고 관리할 수 있도록 해 줌

▶ 웹 페이지 추상화



```
// Header 컴포넌트: 로고와 내비게이션 바
function Header() {
  return (
    <header>
      <h1>My Website</h1>
      <Nav />
    </header>
  );
}
// Navigation 컴포넌트: 웹 사이트 메뉴
function Nav() {
  return (
    <nav>Home | About | Services | Contact</nav>
  );
}
// Article 컴포넌트: 주요 콘텐츠
function Article() {
  return (
    <article>
      <h2>Main Article</h2>
    </article>
  );
}
// Section 컴포넌트: 세부 콘텐츠
function Section() {
  return (
    <section>
      <h3>More Details</h3>
    </section>
  );
```

```
  }
  // Aside 컴포넌트: 광고, 추가 정보
  function Aside() {
    return (
      <aside>
        <h3>Related Links</h3>
      </aside>
    );
  }
  // Footer 컴포넌트: 저작권 정보, 연락처
  function Footer() {
    return (
      <footer>
        <p>© 2025 My Website. All rights reserved.</p>
      </footer>
    );
  }
  // App 컴포넌트: 모든 컴포넌트를 조합해 전체 페이지 구성
  function App() {
    return (
      <div>
        <Header />
          <Article />
          <Aside />
        <Footer />
      </div>
    );
  }
```

3.1 컴포넌트란

- 컴포넌트 단위로 구성할 때의 이점
  1. 재사용 가능
  2. 유지보수 용이
  3. 로직 분리 가능
  4. 복잡한 상태 관리 가능

3.1 컴포넌트란

- 컴포넌트 단위로 구성할 때의 이점

  1. 재사용 가능

  - 컴포넌트의 가장 큰 장점 중 하나는 한 번 만든 UI와 기능을 여러 곳에서 다시 사용할 수 있다는 점

  ✓예를 들어 버튼이 여러 화면에서 반복해서 필요하다고 가정

  - 매번 HTML과 스타일을 새로 작성하는 대신, Button 컴포넌트를 한 번 만들어 두면 여러 곳에서 재사용 가능

3.1 컴포넌트란

- 컴포넌트 단위로 구성할 때의 이점

1. 재사용 가능

```
function Button({ text }) {
  return <button>{text}</button>
}
```

```
<Button text="저장" />
<Button text="삭제" />
<Button text="수정" />
```

이 방식의 장점

- 같은 형태의 UI를 반복 작성하지 않아도 됨
- 디자인 통일이 쉬움 / 수정이 필요할 때 한 곳만 고치면 됨

✓ 예를 들어 버튼 스타일을 바꾸고 싶다면 Button 컴포넌트만 수정하면 됨. 프로젝트 전체에서 사용하는 버튼이 모두 일관되게 바뀜

3.1 컴포넌트란

- 컴포넌트 단위로 구성할 때의 이점

  2. 유지보수 용이
     - React에서 컴포넌트 단위 설계는 유지보수성을 크게 높임
     - 기존 기능 수정
     - 오류 수정
     - 디자인 변경
     - 새로운 기능 추가

3.1 컴포넌트란

- **컴포넌트 단위로 구성할 때의 이점**

  3. 로직 분리 가능

  - 컴포넌트 단위 설계는 화면만 분리하는 것이 아니라 **기능과 로직도 역할별로 분리**
  - **장점**
    - **코드의 책임이 명확해짐**
    - **테스트하기 쉬움**
    - **같은 로직을 다른 컴포넌트에서도 재사용 가능**
    - **디버깅이 쉬워짐**

    ✓**React에서는 이런 흐름이 특히 중요**
    왜냐하면 React는 UI를 컴포넌트로 나누는 구조이면서 동시에 상태, 이벤트, 렌더링 로직을 함께 다루는 방식이기 때문

3.1 컴포넌트란

- **컴포넌트 단위로 구성할 때의 이점**

  4. 복잡한 상태 관리 가능

- **React에서 상태(state)는 매우 중요한 개념. 상태란 화면에 영향을 주는 데이터.**
- **예:**
  - **로그인 여부 / 장바구니 목록 / 입력값 / 체크박스 선택 여부**
  - **모달 열림/닫힘**
  - **서버에서 받아온 데이터**

  ✓**작은 프로젝트에서는 상태 관리가 단순할 수 있지만, 화면이 커지고 기능이 많아지면 상태가 매우 복잡해 짐**

  ✓**컴포넌트 단위 구조는 단순한 분할이 아니라 복잡한 상태를 작은 책임 단위로 나누고, 필요한 범위에서 안전하게 관리할 수 있게 해주는 기반 구조**

3.1 컴포넌트란

- 컴포넌트 나누기: 목적에 맞게 적절한 수준으로 나눌 것
  - 너무 세분하면 관리가 어렵고
  - 너무 뭉뚱그리면 재사용성과 유지보수성이 떨어진다

3.2 컴포넌트의 종류

3.2.1 클래스 컴포넌트

- **클래스 컴포넌트(class component): ES6에서 도입된 클래스 문법을 사용해 컴포넌트 정의**

형식

```
import { Component } from 'react';

class [컴포넌트_이름] extends Component {
  // JSX를 반환하는 render() 메서드
  render() {
    return (...)
  }
}
export default [컴포넌트_이름];
```

3.1 컴포넌트란

- ES6 == ECMAScript 2015
  - ES6 은 React에서 많이 사용하는 현대 JavaScript 문법(ES6) 기반 코드 스타일
- ECMAScript 는 JavaScript의 표준 규격(specification)
- JavaScript 언어의 문법, 기능, 동작 방식을 정의한 표준
- 관리 기관: Ecma International
- JavaScript 엔진들은 이 표준을 기반으로 구현됨 (예: V8 JavaScript engine)
- ECMA = European Computer Manufacturers Association : IT 및 컴퓨팅 기술 표준을 만드는 국제 표준화 기구

3.2 컴포넌트의 종류

3.2.2 함수형 컴포넌트

- **함수형 컴포넌트(functional component): 함수를 사용해 컴포넌트를 작성하는 방식**

형식

```
export default function [컴포넌트_이름]() {
  return (...); // JSX 반환
}
```

- **함수형 컴포넌트의 이름은 파스칼 케이스로 작성**

3.2 컴포넌트의 종류

3.2.3 클래스 컴포넌트와 함수형 컴포넌트 사용

- 클래스 컴포넌트는 문법이 복잡하고, 상태 관리나 코드 재사용이 어렵다는 단점
- 단순히 function 키워드로 컴포넌트를 정의할 수 있어서 자바스크립트를 익힌 사람에게 더 직관적
- 리액트 훅의 도입으로 상태 관리, 생명주기 처리 등 대부분의 기능을 함수형 컴포넌트에서 구현할 수 있게 됨
- 리액트 16.8이 발표된 2019년 이후 대부분 프로젝트에서는 함수형 컴포넌트를 주로 사용

3.3 컴포넌트 기초

3.3.1 컴포넌트 확장자

- 확장자에 따라 JSX 문법을 사용할 수 있는지 여부가 달라짐

▶ CRA와 Vite 기반 프로젝트의 확장자별 JSX 지원 여부

확장자	CRA	Vite
.js	JSX 지원	JSX 지원
.jsx	JSX 지원	JSX 지원
.ts	JSX 지원 안 함	JSX 지원 안 함
.tsx	JSX 지원	JSX 지원

3.3 컴포넌트 기초

3.3.2 컴포넌트 정의

●함수형 컴포넌트 정의

- 함수 선언문 방식: 컴포넌트는 function 키워드로 정의하고 JSX를 return 문 안에 작성
- 컴포넌트 내보내기 방식에 따른 불러오기 차이
  - export 방식: 불러오는 쪽에서 중괄호({})를 사용해 불러와야 함
  - export default 방식: 중괄호 없이 바로 불러올 수 있음 (초보자 권장)

●다른 정의 방식

- 함수 표현식
- 화살표 함수(arrow function)

3.3 컴포넌트 기초

3.3.3 컴포넌트 추가

- 화면에 여러 컴포넌트를 함께 렌더링하는 방법을 익히는 것이 중요
- 새로운 컴포넌트(App 컴포넌트)를 하나 만들어 함께 표시
- 같은 방식으로 Main과 Footer 컴포넌트도 추가

3.3 컴포넌트 기초

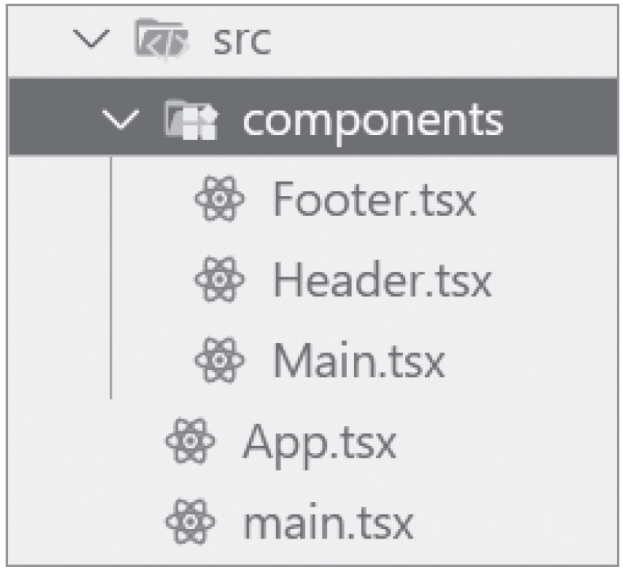
●컴포넌트 파일 분리

- App.tsx 파일 안에 만든 Header, Main, Footer 컴포넌트는 파일 단위로 분리

- 다른 파일에 있는 컴포넌트를 사용할 때

형식
```
import 컴포넌트_이름 from '파일_경로';
```

▶ 추가한 폴더와 파일 구조

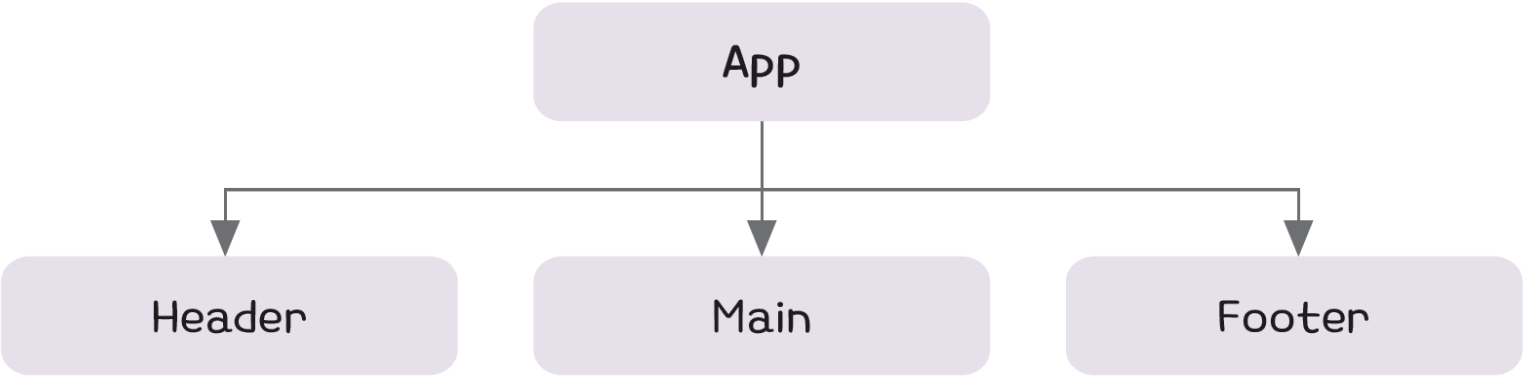


3.3 컴포넌트 기초

3.3.4 컴포넌트 트리

- 컴포넌트 트리(component tree): 하나의 컴포넌트 안에 또 다른 컴포넌트를 포함하는 방식으로 형성된 계층적 구조
- 루트 컴포넌트(root component): 리액트 애플리케이션에서 가장 바깥쪽에 위치하는 컴포넌트

▶ 컴포넌트 트리



3.3 컴포넌트 기초

- 리액트 컴포넌트의 주요 특징
  - 계층 구조: 부모-자식 관계의 계층 구조
  - 단방향 데이터 흐름: 데이터가 항상 부모에서 자식으로만 흐름
  - 재사용성: 한 번 정의한 컴포넌트는 여러 위치에서 반복해서 사용할 수 있음
  - 상태 관리: 각 컴포넌트는 자신만의 상태(state)를 가짐
  - 렌더링 최적화: 가상 DOM을 사용해 실제 DOM과의 차이를 비교한 뒤 변경된 부분만 최소한으로 업데이트

3.4 컴포넌트와 props 객체

3.4.1 props 객체란

- **props(properties): 컴포넌트를 마치 HTML 태그처럼 사용해 값을 속성 형태로 전달**
- **자식 컴포넌트는 props 객체를 통해 부모 컴포넌트가 전달한 데이터를 받아 사용할 수 있음**

3.4.2 props 객체 타입 알아내기

- **전달할 데이터가 많아질 때, 하나의 객체나 배열로 묶어 한 번에 전달하는 것이 더 깔끔하고 효율적**
- **객체 하나를 props로 전달하면 필요한 데이터를 그 안에서 꺼내 사용할 수 있음**

3.4 컴포넌트와 props 객체

3.4.3 props 객체의 구조 분해 할당과 타입 정의

- 매개변수에서의 구조 분해 할당
  - 구조 분해 할당(destructuring assignment): 객체나 배열의 속성을 추출해 변수에 할당하는 자바스크립트 문법
- 함수 내부에서의 구조 분해 할당
  - 함수 내부에서 분해해도 같은 효과
- 타입 정의
  - 함수 매개변수의 타입이 너무 길고 복잡해 보인다면, 타입스크립트의 인터페이스(interface)를 사용해 props 타입을 따로 정의

3.4 컴포넌트와 props 객체

3.4.4 props와 전개 연산자

- 전개 연산자(spread operator): userObj 객체의 속성을 각각 전달하고 싶다면, 전개 연산자(...)를 사용

3.4.5 children

- 컴포넌트를 시작 태그와 종료 태그를 함께 사용하는 형태(<컴포넌트>내용</컴포넌트>)로도 사용할 수 있음

3.5 컴포넌트와 이벤트

3.5.1 이벤트 속성

- **이벤트(event): 사용자와의 상호작용에 따라 발생하는 일련의 사건**

형식

```
<JSXElement 이벤트_속성='이벤트_핸들러'/>
```

▶ 리액트의 주요 이벤트 속성

이벤트 유형	이벤트 속성	설명
마우스 이벤트	onClick ★	클릭 이벤트 발생 시 호출
	onDoubleClick	더블 클릭 이벤트 발생 시 호출
	onMouseEnter	마우스가 요소 위로 올라갔을 때 호출
	onMouseLeave	마우스가 요소에서 이동했을 때 호출
	onMouseMove	마우스가 요소 위에서 움직일 때 호출

계속

3.5 컴포넌트와 이벤트

이벤트 유형	이벤트 속성	설명
마우스 이벤트	onContextMenu	마우스 오른쪽 버튼 클릭 시 호출
	onDrag	드래그할 때 호출
	onDrop	드래그한 요소를 놓을 때 호출
	onMouseDown	마우스 버튼을 눌렀을 때 호출
	onMouseUp	마우스 버튼을 눌렀다 뗐을 때 호출

이벤트 유형	이벤트 속성	설명
키보드 이벤트	onKeyDown ★	키를 눌렀을 때 호출
	onKeyUp ★	키를 눌렀다 뗐을 때 호출
	onKeyPress	키를 누르고 있을 때 호출

계속

3.5 컴포넌트와 이벤트

이벤트 유형	이벤트 속성	설명
폼 이벤트	onChange ★	입력 필드의 값이 변경될 때 호출
	onInput	입력 필드에서 입력이 발생할 때 호출
	onFocus	입력 필드에 포커스가 들어올 때 호출
	onBlur	입력 필드에서 포커스가 빠져나갈 때 호출
	onSubmit ★	폼이 제출될 때 호출
터치 이벤트	onTouchStart	터치가 시작될 때 호출
	onTouchMove	터치 중에 움직일 때 호출
	onTouchEnd	터치가 끝났을 때 호출
기타 이벤트	onLoad	이미지나 iframe 등이 로드될 때 호출
	onError	이미지나 iframe 등에서 오류가 발생했을 때 호출
	onScroll	요소가 스크롤될 때 호출

3.5 컴포넌트와 이벤트

●이벤트 속성 사용 시 주의사항

- **HTML 태그에는 DOM 표준 이벤트 속성만 사용할 수 있음**
- **컴포넌트에서는 전달한 props 값을 받아 실제 DOM 요소에 연결할 수 있음**

```
export default function App() {
  return (
    <>
      <button onFive={() => { alert('five'); }}>click</button>
    </>
  );
}
```

존재하지 않는 이벤트 속성 사용으로 오류 발생

3.5 컴포넌트와 이벤트

3.5.2 이벤트 핸들러

- 이벤트 핸들러: 이벤트가 발생했을 때 실행할 함수

● 인라인 핸들러(inline handler)

- 이벤트 속성 안에 함수를 직접 작성하는 방식

● 함수 참조

- 이벤트 핸들러를 컴포넌트 바깥(JSX 바깥)에 따로 정의하고, 해당 함수를 참조해 이벤트 속성에 할당하는 방식

● 이벤트 핸들러의 매개변수

- 이벤트 핸들러에 매개변수를 전달할 때 인라인 핸들러 방식을 사용해야 함

3.5 컴포넌트와 이벤트

3.5.3 이벤트 객체

- 합성 이벤트(synthetic event): 웹 브라우저의 기본 이벤트를 감싸서 만든 자체 이벤트 객체

●이벤트 객체와 인라인 핸들러

- 인라인 핸들러에서는 이벤트 객체를 함수의 매개변수를 직접 받아 사용

3.5 컴포넌트와 이벤트

●이벤트 객체와 함수 참조

- 함수 참조 방식에서는 이벤트 핸들러에 이벤트 객체를 직접 전달하지 않아도 리액트가 자동으로 이벤트 객체를 해당 함수의 첫 번째 매개변수로 전달

●이벤트 객체와 매개변수

- 이벤트 핸들러에 이벤트 객체와 추가 매개변수를 모두 전달해야 하는 경우 인라인 핸들러 방식을 사용해 이벤트 객체와 원하는 매개변수를 명시적으로 전달해야 함

3.5 컴포넌트와 이벤트

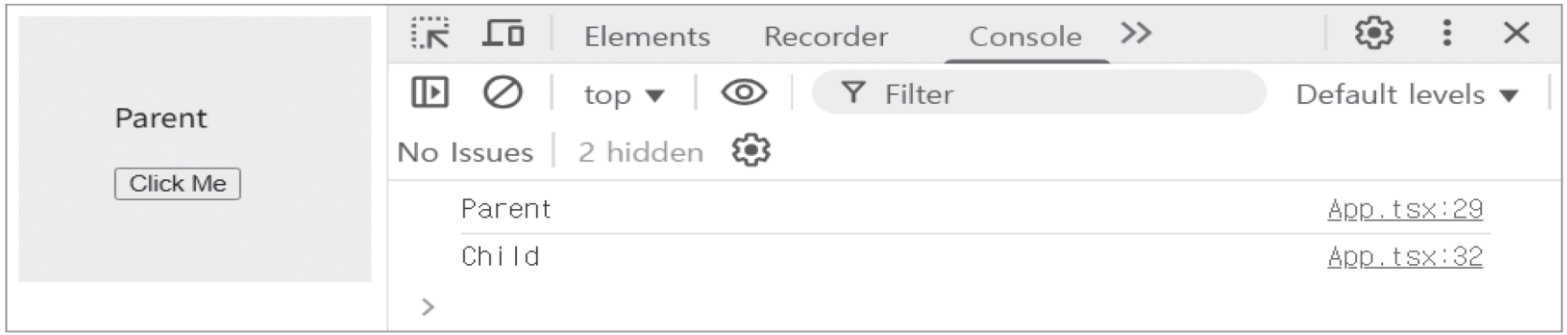
3.5.4 이벤트 전파(event propagation)

- 이벤트가 발생했을 때 부모 또는 자식 요소로 전달되는 현상

●캡처링(capturing)

- 이벤트가 부모 요소에서 시작해 자식 요소로 내려가는 방식

▶ 캡처링 결과

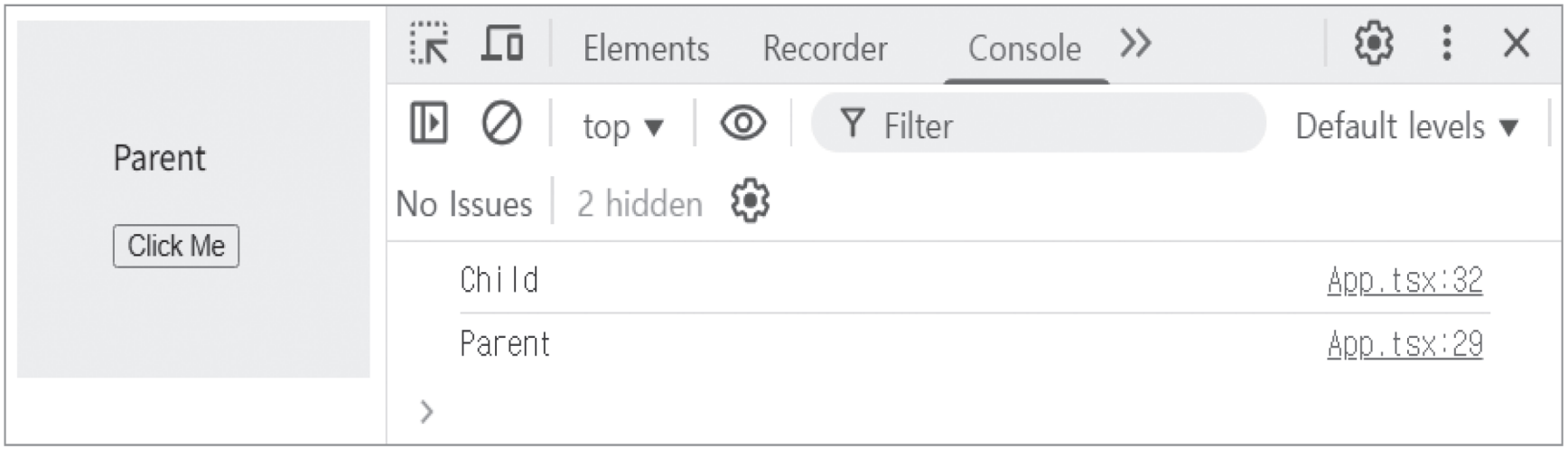


3.5 컴포넌트와 이벤트

●버블링(bubbling)

- 이벤트가 자식 요소에서 시작해 부모 요소로 전파되는 방식

▶ 버블링 결과

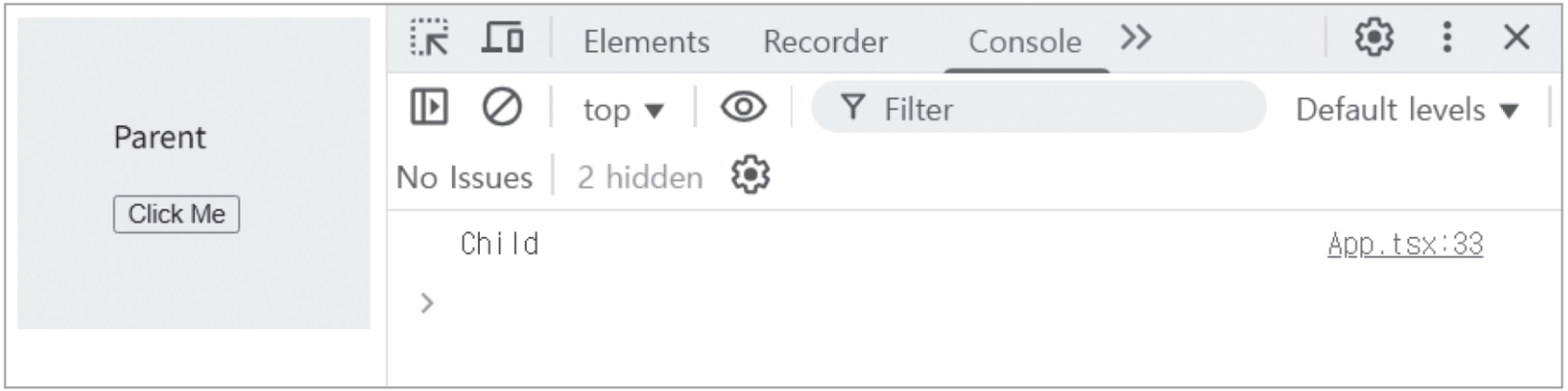


3.5 컴포넌트와 이벤트

●이벤트 전파 막기

- 이벤트 객체의 stopPropagation() 메서드를 사용해 버블링을 차단

▶ stopPropagation() 메서드 호출 결과



●이벤트 기본 동작 막기

- 이벤트 객체의 preventDefault() 메서드를 사용해 기본 동작 막기